

```
In [ ]: import marimo as mo
```

VQ-VAE Tokenizer Evaluation

Protein / Ligand VQ-VAE の学習後の定量的性能評価ノートブック。

評価項目:

1. 再構成誤差 (MSE) — 全体 & 次元別
2. Codebook 利用率 & Perplexity
3. Original vs Reconstructed の散布図
4. Codebook 使用頻度の分布
5. 潜在空間の可視化 (t-SNE)

```
In [ ]: import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch

# Add project root to path (resolve from this file, not cwd)
project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

from src.config import CrossDockedConfig, VQVAETrainingConfig
from src.data.descriptors import ComplexDescriptorDataModule
from src.model.vqvae_module import VQVAEModule

# '%matplotlib inline' command supported automatically in marimo
plt.rcParams["figure.dpi"] = 120
# Bump global font sizes so plots stay legible when pasted into slides.
plt.rcParams.update({
    "font.size": 14,
    "axes.titlesize": 16,
    "axes.labelsize": 14,
    "xtick.labelsize": 12,
    "ytick.labelsize": 12,
    "legend.fontsize": 12,
    "figure.titlesize": 18,
})
```

1. Load checkpoint and data

```
In [ ]: # --- Checkpoint path (edit here) ---
# Best checkpoint is typically the one with lowest val/protein_recon.
# List available checkpoints:
```

```

ckpt_dir = project_root / "checkpoints"
if ckpt_dir.exists():
    ckpts = sorted(ckpt_dir.rglob("*.ckpt"))
    for _p in ckpts:
        print(_p.relative_to(project_root))
else:
    print("No checkpoints/ directory found. Check wandb run directory.")
    ckpts = sorted(project_root.rglob("*.ckpt"))
    for _p in ckpts[-5:]:
        print(_p.relative_to(project_root))

```

No checkpoints/ directory found. Check wandb run directory.
pocket-ligand-vqvae/uhhyc6y7/checkpoints/vqvae-epoch=86-val/protein_recon=0.1287.ckpt
pocket-ligand-vqvae/uhhyc6y7/checkpoints/vqvae-epoch=93-val/protein_recon=0.1288.ckpt
pocket-ligand-vqvae/yggua4f0/checkpoints/vqvae-epoch=90-val/protein_recon=0.1316.ckpt
pocket-ligand-vqvae/yggua4f0/checkpoints/vqvae-epoch=95-val/protein_recon=0.1316.ckpt
pocket-ligand-vqvae/yggua4f0/checkpoints/vqvae-epoch=99-val/protein_recon=0.1316.ckpt

```

In [ ]: import os

# Override via `VQVAE_CKPT` env var so a single notebook can be exported
# per run without editing source in between. Falls back to the baseline
# (yggua4f0) used in the original evaluation.
CKPT_PATH = os.environ.get(
    "VQVAE_CKPT",
    "/gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/uhhyc6y7/checkpoints/vqvae-epoch=93-val/protein_recon=0.1288.ckpt"
)

print(f>Loading: {CKPT_PATH}")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
module = VQVAEModule.load_from_checkpoint(str(CKPT_PATH), map_location=device)
module.eval()
module.to(device)

protein_vqvae = module.protein_vqvae
ligand_vqvae = module.ligand_vqvae
print(f>Device: {device}")

```

Loading: /gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/pocket-ligand-vqvae/29sx3ezx/checkpoints/vqvae-epoch=98-val/protein_recon=0.1197.ckpt

```

/gs/bs/tga-ohuelab/sakano/git/pocket-conditioned-ligand-gen/.venv/lib/python
3.12/site-packages/torch/nn/modules/transformer.py:392: UserWarning: enable_
nested_tensor is True, but self.use_nested_tensor is False because encoder_l
ayer.norm_first was True
  warnings.warn(
Device: cuda

```

```

In [ ]: # Load cached descriptors and prepare test split
import os as _os
from pathlib import Path as _Path

```

```

config = VQVAETrainingConfig()
data_config = CrossDockedConfig(data_dir=project_root / "data")
dm = ComplexDescriptorDataModule(config, data_config)
# Override the cache dir so B2 (v2 cache + skip_sincos stats) can be
# evaluated against the matching shards/stats. Defaults to the v1 cache.
_cache_override = _os.environ.get("VQVAE_CACHE_DIR")
if _cache_override:
    dm.cache_dir = _Path(_cache_override)
dm.setup()

norm_stats = dm.norm_stats

# The full test split is ~2M complexes across 436 shards; loading it all
# takes the better part of an hour and is overkill for MSE / t-SNE
# statistics. Cap the notebook at MAX_TEST_COMPLEXES by stopping early.
MAX_TEST_COMPLEXES = 2000

if dm.protein_test is None:
    prot_mean_np = norm_stats["protein_mean"].numpy()
    prot_std_np = norm_stats["protein_std"].numpy()
    lig_mean_np = norm_stats["ligand_mean"].numpy()
    lig_std_np = norm_stats["ligand_std"].numpy()

    protein_test_pockets = []
    ligand_test_molecules = []
    # Read each test shard once and pull out both protein + ligand,
    # so we don't pay 2x disk I/O across two separate iterators.
    for shard_idx, local_indices in dm._test_plan:
        if len(protein_test_pockets) >= MAX_TEST_COMPLEXES:
            break
        shard_path = dm._shard_dir / f"shard_{shard_idx:04d}.pt"
        shard_data = torch.load(shard_path, weights_only=False)
        for local_idx in local_indices:
            if len(protein_test_pockets) >= MAX_TEST_COMPLEXES:
                break
            cplx = shard_data[local_idx]
            protein_test_pockets.append(
                torch.from_numpy(
                    (cplx["protein"] - prot_mean_np) / prot_std_np,
                )
                .float()
                .to(device),
            )
            ligand_test_molecules.append(
                torch.from_numpy(
                    (cplx["ligand"] - lig_mean_np) / lig_std_np,
                )
                .float()
                .to(device),
            )
        del shard_data
    else:
        protein_test_pockets = [
            t.to(device) for t in dm.protein_test[:MAX_TEST_COMPLEXES]
        ]

```

```

ligand_test_molecules = [
    t.to(device) for t in dm.ligand_test[:MAX_TEST_COMPLEXES]
]

protein_test_flat = torch.cat(protein_test_pockets)
ligand_test_flat = torch.cat(ligand_test_molecules)

print(
    f"Protein test: {len(protein_test_pockets)} pockets, "
    f"{protein_test_flat.shape[0]} residues total"
)
print(
    f"Ligand test: {len(ligand_test_molecules)} molecules, "
    f"{ligand_test_flat.shape[0]} atoms total"
)

```

types-file path: using legacy random 80/10/10 split; manifest-fold split unavailable

Protein test: 2000 pockets, 54293 residues total

Ligand test: 2000 molecules, 55574 atoms total

```

In [ ]: # Both protein and ligand share the same TransformerVQVAE architecture and
# consume variable-length per-pocket / per-molecule sequences.
@torch.no_grad()
def run_vqvae(model: "TransformerVQVAE", sequences: "list[Tensor]"):
    """Run TransformerVQVAE per-sequence and collect recon/indices/z."""
    all_recon, all_indices, all_z = [], [], []
    for seq in sequences:
        if seq.shape[0] == 0:
            continue
        x_seq = seq.unsqueeze(0) # (1, N, D)
        h = model.input_proj(model.input_norm(x_seq))
        h = h + model.pos_encoding[: seq.shape[0]]
        h = model.transformer_encoder(h)
        z = model.latent_norm(model.latent_proj(h)).squeeze(0) # (N, latent_dim)
        quantized, indices, _ = model.codebook(z)
        q_seq = model.latent_unproj(quantized).unsqueeze(0)
        dec_in = q_seq + model.pos_encoding[: seq.shape[0]]
        dec_out = model.transformer_decoder(dec_in)
        recon = model.output_proj(dec_out).squeeze(0) # (N, D)
        all_recon.append(recon)
        all_indices.append(indices)
        all_z.append(z)
    return torch.cat(all_recon), torch.cat(all_indices), torch.cat(all_z)

prot_recon, prot_indices, prot_z = run_vqvae(protein_vqvae, protein_test_pockets)
lig_recon, lig_indices, lig_z = run_vqvae(ligand_vqvae, ligand_test_molecules)

print("Inference done.")
print(
    f"Protein: recon {prot_recon.shape}, indices {prot_indices.shape}, z {prot_z.shape}"
)
print(
    f"Ligand: recon {lig_recon.shape}, indices {lig_indices.shape}, z {lig_z.shape}"
)

```

Inference done.

Protein: recon torch.Size([54293, 12]), indices torch.Size([54293]), z torch.Size([54293, 16])

Ligand: recon torch.Size([55574, 4]), indices torch.Size([55574]), z torch.Size([55574, 8])

2. Reconstruction Error (MSE)

正規化空間と元スケールの両方で再構成誤差を評価する。

```
In [ ]: def compute_mse_metrics(
    original: "Tensor",
    reconstructed: "Tensor",
    name: str,
    norm_mean: "Tensor",
    norm_std: "Tensor",
):
    """Compute MSE in both normalized and original scale."""
    orig_np = original.cpu().numpy()
    recon_np = reconstructed.cpu().numpy()

    # Normalized space
    mse_norm = np.mean((orig_np - recon_np) ** 2)
    per_dim_mse_norm = np.mean((orig_np - recon_np) ** 2, axis=0)

    # De-normalize to original scale
    mean_np = norm_mean.numpy()
    std_np = norm_std.numpy()
    orig_denorm = orig_np * std_np + mean_np
    recon_denorm = recon_np * std_np + mean_np
    mse_orig = np.mean((orig_denorm - recon_denorm) ** 2)
    per_dim_mse_orig = np.mean((orig_denorm - recon_denorm) ** 2, axis=0)

    print(f"=== {name} VQ-VAE ===")
    print(f" Overall MSE (normalized): {mse_norm:.6f}")
    print(f" Overall MSE (original): {mse_orig:.6f}")
    print(f" Overall RMSE (original): {np.sqrt(mse_orig):.6f}")
    print()

    return {
        "mse_norm": mse_norm,
        "per_dim_mse_norm": per_dim_mse_norm,
        "mse_orig": mse_orig,
        "per_dim_mse_orig": per_dim_mse_orig,
        "orig_denorm": orig_denorm,
        "recon_denorm": recon_denorm,
    }

prot_metrics = compute_mse_metrics(
    protein_test_flat,
    prot_recon,
    "Protein",
    norm_stats["protein_mean"],
    norm_stats["protein_std"],
```

```
)
lig_metrics = compute_mse_metrics(
    ligand_test_flat,
    lig_recon,
    "Ligand",
    norm_stats["ligand_mean"],
    norm_stats["ligand_std"],
)
```

=== Protein VQ-VAE ===

```
Overall MSE (normalized):  0.127916
Overall MSE (original):    0.086713
Overall RMSE (original):   0.294471
```

=== Ligand VQ-VAE ===

```
Overall MSE (normalized):  0.017901
Overall MSE (original):    0.009797
Overall RMSE (original):   0.098979
```

2.1 Per-dimension MSE

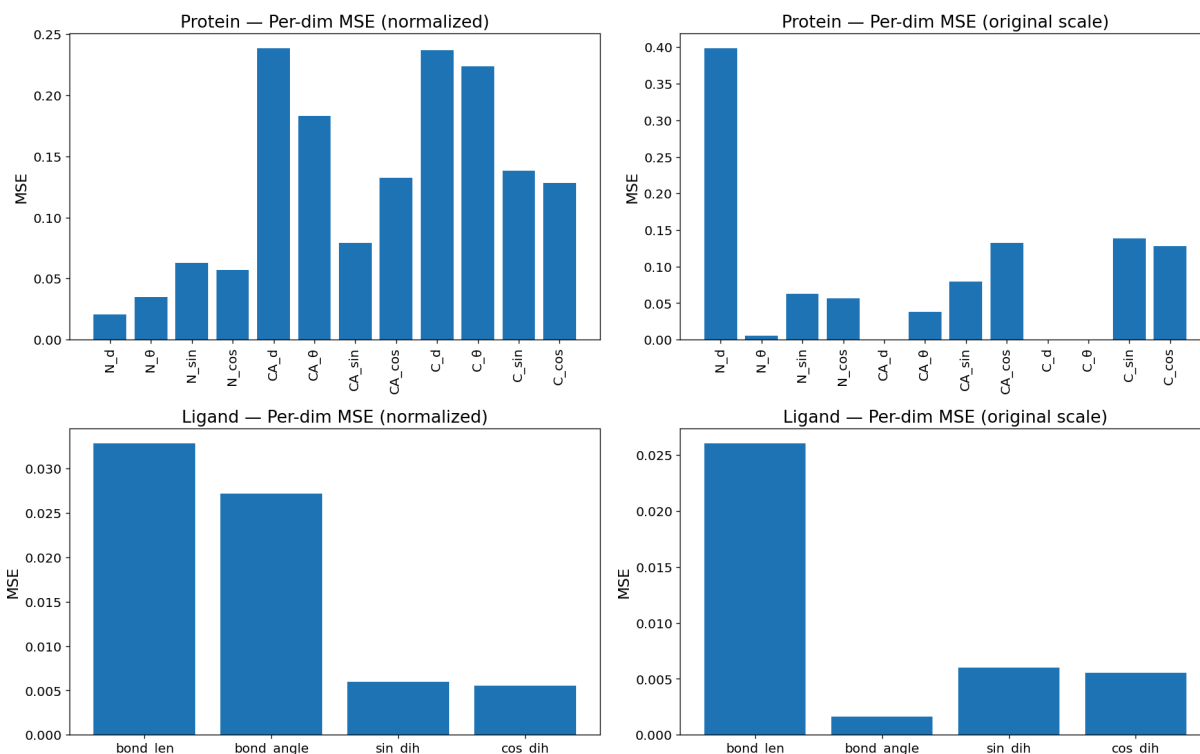
Protein: 12-D backbone Z-matrix (4 values $\times$ 3 atoms N/CA/C). 4 values per atom are (bond_length, bond_angle, sin_torsion, cos_torsion) for continuation residues, or pocket-frame-anchored spherical coords for segment-start residues. Ligand: dim 0 = bond length, dim 1 = bond angle, dim 2-3 = sin/cos dihedral

```
In [ ]: _fig, _axes = plt.subplots(2, 2, figsize=(16, 10))
prot_dim_labels = [
    "N_d",
    "N_theta",
    "N_sin",
    "N_cos",
    "CA_d",
    "CA_theta",
    "CA_sin",
    "CA_cos",
    "C_d",
    "C_theta",
    "C_sin",
    "C_cos",
]
n_prot_dims = len(prot_dim_labels)
# Protein - normalized
_axes[0, 0].bar(range(n_prot_dims), prot_metrics["per_dim_mse_norm"])
_axes[0, 0].set_xticks(range(n_prot_dims), prot_dim_labels, rotation=90, for
_axes[0, 0].set_title("Protein - Per-dim MSE (normalized)")
_axes[0, 0].set_ylabel("MSE")
_axes[0, 1].bar(range(n_prot_dims), prot_metrics["per_dim_mse_orig"])
_axes[0, 1].set_xticks(range(n_prot_dims), prot_dim_labels, rotation=90, for
# Protein - original scale
_axes[0, 1].set_title("Protein - Per-dim MSE (original scale)")
_axes[0, 1].set_ylabel("MSE")
```

```

lig_dim_labels = ["bond_len", "bond_angle", "sin_dih", "cos_dih"]
_axes[1, 0].bar(range(4), lig_metrics["per_dim_mse_norm"])
_axes[1, 0].set_xticks(range(4), lig_dim_labels, rotation=0, fontsize=12)
# Ligand - normalized
_axes[1, 0].set_title("Ligand - Per-dim MSE (normalized)")
_axes[1, 0].set_ylabel("MSE")
_axes[1, 1].bar(range(4), lig_metrics["per_dim_mse_orig"])
_axes[1, 1].set_xticks(range(4), lig_dim_labels, rotation=0, fontsize=12)
_axes[1, 1].set_title("Ligand - Per-dim MSE (original scale)")
_axes[1, 1].set_ylabel("MSE")
# Ligand - original scale
_fig.tight_layout()
_fig

```



3. Codebook Utilization & Perplexity

理想的なcodebookは全コードが均等に使われる。

- **Utilization:** 使用されたコード数 / 全コード数 (1.0が理想)
- **Perplexity:** $\exp(\text{entropy})$ 。均等利用時は codebook_size と一致

```

In [ ]: def codebook_stats(indices: "Tensor", codebook_size: int, name: str):
        """Compute and print codebook utilization metrics."""
        idx_np = indices.cpu().numpy()
        unique = np.unique(idx_np)
        utilization = len(unique) / codebook_size

        counts = np.bincount(idx_np, minlength=codebook_size).astype(float)
        probs = counts / counts.sum()
        probs_nonzero = probs[probs > 0]
        entropy = -np.sum(probs_nonzero * np.log(probs_nonzero))

```

```

perplexity = np.exp(entropy)

# Max perplexity = codebook_size (uniform distribution)
max_perplexity = codebook_size
normalized_perplexity = perplexity / max_perplexity

print(f"=== {name} Codebook ===")
print(f"  Codebook size:           {codebook_size}")
print(f"  Active codes:              {len(unique)}")
print(f"  Utilization:                {utilization:.4f}")
print(f"  Perplexity:                 {perplexity:.1f} / {max_perplexity}")
print(f"  Normalized perplexity:     {normalized_perplexity:.4f}")
print(f"  Dead codes:                 {codebook_size - len(unique)}")
print()
return counts

prot_counts = codebook_stats(prot_indices, config.protein.codebook_size, "Protein")
lig_counts = codebook_stats(lig_indices, config.ligand.codebook_size, "Ligand")

=== Protein Codebook ===
  Codebook size:           2048
  Active codes:            2035
  Utilization:             0.9937
  Perplexity:              1606.7 / 2048
  Normalized perplexity:   0.7845
  Dead codes:              13

=== Ligand Codebook ===
  Codebook size:           1024
  Active codes:            1022
  Utilization:             0.9980
  Perplexity:              759.0 / 1024
  Normalized perplexity:   0.7412
  Dead codes:              2

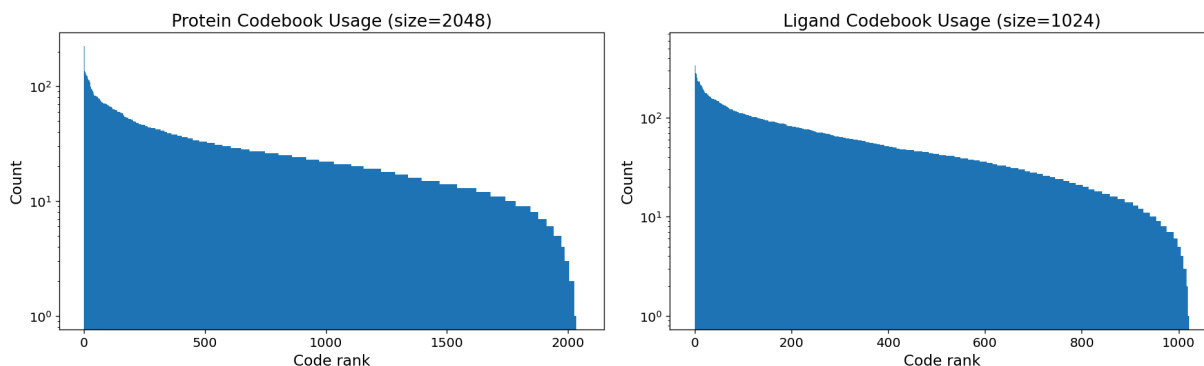
```

3.1 Codebook usage distribution

```

In [ ]: _fig, _axes = plt.subplots(1, 2, figsize=(16, 5))
prot_sorted = np.sort(prot_counts)[::-1]
# Protein codebook usage - sorted descending
_axes[0].bar(range(len(prot_sorted)), prot_sorted, width=1.0)
_axes[0].set_xlabel("Code rank")
_axes[0].set_ylabel("Count")
_axes[0].set_title(f"Protein Codebook Usage (size={config.protein.codebook_size})")
_axes[0].set_yscale("log")
lig_sorted = np.sort(lig_counts)[::-1]
_axes[1].bar(range(len(lig_sorted)), lig_sorted, width=1.0)
# Ligand codebook usage - sorted descending
_axes[1].set_xlabel("Code rank")
_axes[1].set_ylabel("Count")
_axes[1].set_title(f"Ligand Codebook Usage (size={config.ligand.codebook_size})")
_axes[1].set_yscale("log")
_fig.tight_layout()
_fig

```

4. Original vs Reconstructed scatter plots

各次元で元の値 vs 再構成値をプロットし、対角線からのずれを確認する。元スケールで表示。

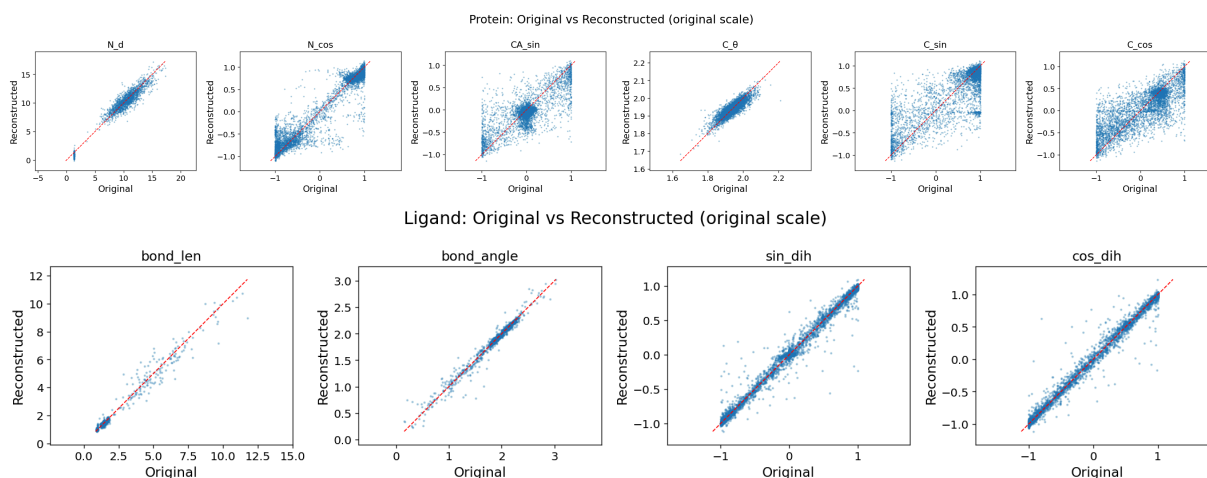
```
In [ ]: def scatter_orig_vs_recon(
    orig: "ndarray",
    recon: "ndarray",
    dim_labels: list[str],
    title: str,
    max_points: int = 5000,
):
    """Scatter plot of original vs reconstructed for selected dimensions."""
    n_dims = orig.shape[1]
    max_inline_dims = 8
    if n_dims <= max_inline_dims:
        selected = list(range(n_dims))
    else:
        selected = [
            0,
            n_dims // 4,
            n_dims // 2,
            3 * n_dims // 4,
            n_dims - 2,
            n_dims - 1,
        ]
    n_plots = len(selected)
    _fig, _axes = plt.subplots(1, n_plots, figsize=(4.5 * n_plots, 4.2))
    if n_plots == 1:
        _axes = [_axes]
    rng = np.random.default_rng(42)
    idx = rng.choice(len(orig), min(max_points, len(orig)), replace=False)
    for ax, dim in zip(_axes, selected, strict=False):
        ax.scatter(orig[idx, dim], recon[idx, dim], s=2, alpha=0.3)
        lo = min(orig[idx, dim].min(), recon[idx, dim].min())
        hi = max(orig[idx, dim].max(), recon[idx, dim].max())
        ax.plot([lo, hi], [lo, hi], "r--", linewidth=1.0)
        ax.set_xlabel("Original")
        ax.set_ylabel("Reconstructed")
        ax.set_title(dim_labels[dim], fontsize=14)
        ax.set_aspect("equal", adjustable="datalim") # Subsample for plotti
    _fig.suptitle(title, fontsize=18)
```

```

_fig.tight_layout()
plt.show()

scatter_orig_vs_recon(
    prot_metrics["orig_denorm"],
    prot_metrics["recon_denorm"],
    prot_dim_labels,
    "Protein: Original vs Reconstructed (original scale)",
)
scatter_orig_vs_recon(
    lig_metrics["orig_denorm"],
    lig_metrics["recon_denorm"],
    lig_dim_labels,
    "Ligand: Original vs Reconstructed (original scale)",
)

```



5. Reconstruction error distribution

サンプルごとの再構成誤差の分布を確認し、外れ値の存在を把握する。

```

In [ ]: _fig, _axes = plt.subplots(1, 2, figsize=(14, 5))
prot_per_sample = np.mean(
    (protein_test_flat.cpu().numpy() - prot_recon.cpu().numpy()) ** 2, axis=
)
lig_per_sample = np.mean(
    (ligand_test_flat.cpu().numpy() - lig_recon.cpu().numpy()) ** 2, axis=1
)
_axes[0].hist(prot_per_sample, bins=100, edgecolor="none", alpha=0.8)
_axes[0].axvline(
    np.median(prot_per_sample),
    color="r",
    linestyle="--",
    label=f"median={np.median(prot_per_sample):.4f}",
)
_axes[0].axvline(
    np.mean(prot_per_sample),
    color="orange",
    linestyle="--",
    label=f"mean={np.mean(prot_per_sample):.4f}",
)

```

```

)
_axes[0].set_xlabel("Per-sample MSE")
_axes[0].set_ylabel("Count")
_axes[0].set_title("Protein – Per-sample MSE distribution")
_axes[0].legend()
_axes[1].hist(lig_per_sample, bins=100, edgecolor="none", alpha=0.8)
_axes[1].axvline(
    np.median(lig_per_sample),
    color="r",
    linestyle="--",
    label=f"median={np.median(lig_per_sample):.4f}",
)
_axes[1].axvline(
    np.mean(lig_per_sample),
    color="orange",
    linestyle="--",
    label=f"mean={np.mean(lig_per_sample):.4f}",
)
_axes[1].set_xlabel("Per-sample MSE")
_axes[1].set_ylabel("Count")
_axes[1].set_title("Ligand – Per-sample MSE distribution")
_axes[1].legend()
_fig.tight_layout()
for name, mse_arr in [("Protein", prot_per_sample), ("Ligand", lig_per_sample)]:
    print(f"{name} per-sample MSE percentiles:")
    for _p in [50, 75, 90, 95, 99]:
        print(f"  P{_p}: {np.percentile(mse_arr, _p):.6f}")
    print()
_fig

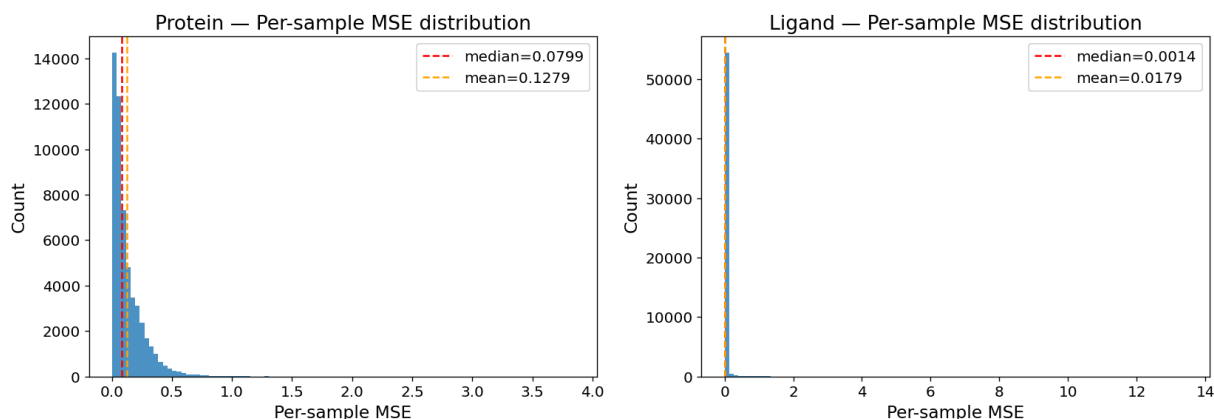
```

Protein per-sample MSE percentiles:

P50: 0.079933
 P75: 0.176270
 P90: 0.296828
 P95: 0.382824
 P99: 0.618472

Ligand per-sample MSE percentiles:

P50: 0.001435
 P75: 0.004140
 P90: 0.012152
 P95: 0.030337
 P99: 0.380271



5.5 3D Reconstruction RMSD (Å)

記述子空間での再構成誤差ではなく、実際の3次元座標に復元した上での RMSD を評価する。

- **Protein:** backbone (N, CA, C) 座標の RMSD
- **Ligand:** 重原子座標の RMSD

2 種類の RMSD を並べて報告する:

- **Per-atom (superposition なし):** pocket frame を共有しているため、segment-start のズレや NeRF 累積による剛体シフトがそのまま出る。生成 pipeline 全体の精度。
- **Kabsch-aligned:** 原子集合を剛体 (回転 + 並進) でベストフィットさせた後の RMSD。内部変形 (conformer shape) の再現精度のみを反映する。

複数の複合体をサンプリングし、VQ-VAE の encode → decode → 記述子逆変換 → 3D座標 のパイプライン全体の精度を確認する。

```
In [ ]: import pyarrow.parquet as pq

from src.config import PocketExtractionConfig
from src.tokenizers.ligand import LigandDescriptor, parse_sdf
from src.tokenizers.protein import (
    BackboneZMatrixDescriptor,
    _compute_canonical_frame,
    extract_pocket,
)

def kabsch_align(p: np.ndarray, q: np.ndarray):
    """Align q onto p with Kabsch; return centered p, aligned q, and RMSD."""
    p_c = p - p.mean(axis=0)
    q_c = q - q.mean(axis=0)
    h = q_c.T @ p_c
    u, _, vt = np.linalg.svd(h)
    d = np.sign(np.linalg.det(vt.T @ u.T))
    rot = vt.T @ np.diag([1.0, 1.0, d]) @ u.T
    q_aligned = q_c @ rot.T
```

```

    rmsd = float(np.sqrt(np.mean(np.sum((p_c - q_aligned) ** 2, axis=-1))))
    return p_c, q_aligned, rmsd

def kabsch_rmsd(p: np.ndarray, q: np.ndarray) -> float:
    """Per-atom RMSD after optimal rigid-body alignment of q onto p (Kabsch)
    return kabsch_align(p, q)[2]

pocket_config = PocketExtractionConfig()
protein_desc_calc = BackboneZMatrixDescriptor()
ligand_desc_calc = LigandDescriptor()
# Use the HuggingFace hub cache manifest instead of the CrossDocked2020
# types/ tree (which isn't present on this machine).
hub_cache_dir = project_root / "data" / "hub_cache"
manifest_path = hub_cache_dir / "repo" / "manifest.parquet"
receptor_dir = hub_cache_dir / "receptors"
ligand_dir = hub_cache_dir / "ligands"
manifest_df = pq.read_table(manifest_path).to_pandas()
# Filter to the cdonly test split for fold 0 – matches the original
# cdonly_*_test0.types file that this cell used to read.
test_df = manifest_df[
    (manifest_df["source_type"] == "cdonly")
    & (manifest_df["cdonly_fold0"] == "test")
].reset_index(drop=True)
entries = [
    (
        f"{row.complex_dir}/{row.receptor_pdb}",
        f"{row.pair_idx:07d}.sdf.gz",
        0,
    )
    for row in test_df.itertuples(index=False)
]
prot_mean = norm_stats["protein_mean"].to(device)
prot_std = norm_stats["protein_std"].to(device)
lig_mean = norm_stats["ligand_mean"].to(device)
lig_std = norm_stats["ligand_std"].to(device)
N_SAMPLES = 2000
rng = np.random.default_rng(42)
sample_indices = rng.choice(
    len(entries), size=min(N_SAMPLES * 5, len(entries)), replace=False
)
prot_rmsd_list = []
prot_rmsd_aligned_list = []
lig_rmsd_list = []
lig_rmsd_aligned_list = []
# Joint (protein backbone + ligand heavy atoms aligned together) – measures
# how well the *relative* protein/ligand pose is preserved end-to-end.
joint_rmsd_list = []
prot_in_joint_rmsd_list = []
lig_in_joint_rmsd_list = []
n_done = 0
for idx in sample_indices:
    if n_done >= N_SAMPLES:
        break
    rec_rel, lig_rel, _pose_idx = entries[idx]
    rec_path = receptor_dir / rec_rel
    lig_path = ligand_dir / lig_rel

```

```

if not rec_path.exists() or not lig_path.exists():
    continue
try:
    molecules = parse_sdf(lig_path)
    if not molecules:
        continue
    mol = molecules[0]
    lig_coords_orig = np.array(
        [(a[1], a[2], a[3]) for a in mol["atoms"] if a[0] != "H"],
        dtype=np.float32,
    )
    pocket_result = extract_pocket(rec_path, lig_coords_orig, pocket_core)
    if pocket_result is None:
        continue
    backbone_coords_orig, _seq, residue_ids = pocket_result
    ca_coords = backbone_coords_orig[:, 1].astype(np.float64)
    centroid, rotation = _compute_canonical_frame(ca_coords)
    pocket_frame = (centroid, rotation)
    prot_desc, prot_meta = protein_desc_calc.compute(
        backbone_coords_orig,
        residue_ids,
        pocket_frame=pocket_frame,
    )
    prot_t = torch.from_numpy(prot_desc).to(device)
    prot_norm = (prot_t - prot_mean) / prot_std
    with torch.no_grad():
        pi = protein_vqvae.encode(prot_norm)
        prot_recon_norm = protein_vqvae.decode(pi)
    prot_recon_desc = (prot_recon_norm * prot_std + prot_mean).cpu().numpy()
    backbone_recon = BackboneZMatrixDescriptor.descriptor_to_backbone_coords(
        prot_recon_desc, prot_meta
    )
    prot_rmsd = np.sqrt(
        np.mean(np.sum((backbone_coords_orig - backbone_recon) ** 2, axis=1))
    )
    prot_flat_orig = backbone_coords_orig.reshape(-1, 3).astype(np.float64)
    prot_flat_recon = backbone_recon.reshape(-1, 3).astype(np.float64)
    prot_rmsd_aligned = kabsch_rmsd(prot_flat_orig, prot_flat_recon)
    lig_desc, _elements, lig_meta = ligand_desc_calc.compute(
        mol["atoms"], mol["bonds"], pocket_frame=pocket_frame
    )
    if len(lig_desc) == 0:
        continue
    lig_t = torch.from_numpy(lig_desc).to(device)
    lig_norm = (lig_t - lig_mean) / lig_std
    with torch.no_grad():
        li = ligand_vqvae.encode(lig_norm)
        lig_recon_norm = ligand_vqvae.decode(li)
    lig_recon_desc = (lig_recon_norm * lig_std + lig_mean).cpu().numpy()
    lig_coords_recon = LigandDescriptor.descriptor_to_coords(
        lig_recon_desc, lig_meta, pocket_frame=pocket_frame
    )
    heavy_atoms = [a for a in mol["atoms"] if a[0] != "H"]
    lig_coords_orig_arr = np.array(
        [(a[1], a[2], a[3]) for a in heavy_atoms], dtype=np.float64
    )

```

```

lig_rmsd = np.sqrt(
    np.mean(
        np.sum((lig_coords_orig_arr - lig_coords_recon) ** 2, axis=-1)
    )
)
lig_rmsd_aligned = kabsch_rmsd(
    lig_coords_orig_arr, lig_coords_recon.astype(np.float64)
)
# Joint-aligned: stack protein backbone + ligand heavy atoms as one
# rigid body, Kabsch-fit, then split the residual to see how much
# of the error sits in each component when the *complex* is best-fit
n_prot_atoms = len(prot_flat_orig)
joint_orig = np.vstack([prot_flat_orig, lig_coords_orig_arr])
joint_recon = np.vstack(
    [prot_flat_recon, lig_coords_recon.astype(np.float64)]
)
joint_orig_c, joint_recon_aligned, joint_rmsd = kabsch_align(
    joint_orig, joint_recon
)
prot_diff_joint = (
    joint_orig_c[:n_prot_atoms] - joint_recon_aligned[:n_prot_atoms]
)
lig_diff_joint = (
    joint_orig_c[n_prot_atoms:] - joint_recon_aligned[n_prot_atoms:]
)
prot_in_joint_rmsd = float(
    np.sqrt(np.mean(np.sum(prot_diff_joint ** 2, axis=-1)))
)
lig_in_joint_rmsd = float(
    np.sqrt(np.mean(np.sum(lig_diff_joint ** 2, axis=-1)))
)
# Append everything atomically: a partial failure above drops the
# whole complex instead of leaving protein/ligand list lengths
# mismatched.
prot_rmsd_list.append(prot_rmsd)
prot_rmsd_aligned_list.append(prot_rmsd_aligned)
lig_rmsd_list.append(lig_rmsd)
lig_rmsd_aligned_list.append(lig_rmsd_aligned)
joint_rmsd_list.append(joint_rmsd)
prot_in_joint_rmsd_list.append(prot_in_joint_rmsd)
lig_in_joint_rmsd_list.append(lig_in_joint_rmsd)
n_done += 1
except Exception: # noqa: BLE001, S112
    continue
prot_rmsd_arr = np.array(prot_rmsd_list)
prot_rmsd_aligned_arr = np.array(prot_rmsd_aligned_list)
lig_rmsd_arr = np.array(lig_rmsd_list)
lig_rmsd_aligned_arr = np.array(lig_rmsd_aligned_list)
joint_rmsd_arr = np.array(joint_rmsd_list)
prot_in_joint_rmsd_arr = np.array(prot_in_joint_rmsd_list)
lig_in_joint_rmsd_arr = np.array(lig_in_joint_rmsd_list)
print(f"Evaluated {len(prot_rmsd_arr)} complexes")
print()

def _print_rmsd_stats(name: str, arr: np.ndarray) -> None:
    print(f"{name}:")

```

```

    print(f"   Mean:    {arr.mean():.4f}")
    print(f"   Median: {np.median(arr):.4f}")
    print(f"   Std:     {arr.std():.4f}")

_print_rmsd_stats("Protein backbone RMSD – per-atom (Å)", prot_rmsd_arr)
print()
_print_rmsd_stats(
    "Protein backbone RMSD – Kabsch-aligned (Å)", prot_rmsd_aligned_arr
)
print()
_print_rmsd_stats("Ligand heavy-atom RMSD – per-atom (Å)", lig_rmsd_arr)
print()
_print_rmsd_stats(
    "Ligand heavy-atom RMSD – Kabsch-aligned (Å)", lig_rmsd_aligned_arr
)

def plot_rmsd_hist(ax, arr: np.ndarray, title: str) -> None:
    ax.hist(arr, bins=50, edgecolor="none", alpha=0.8)
    ax.axvline(
        np.median(arr),
        color="r",
        linestyle="--",
        label=f"median={np.median(arr):.3f} Å",
    )
    ax.axvline(
        arr.mean(),
        color="orange",
        linestyle="--",
        label=f"mean={arr.mean():.3f} Å",
    )
    ax.set_xlabel("RMSD (Å)")
    ax.set_ylabel("Count")
    ax.set_title(title)
    ax.legend()

_fig, _axes = plt.subplots(2, 2, figsize=(14, 10))
plot_rmsd_hist(_axes[0, 0], prot_rmsd_arr, "Protein backbone – per-atom")
plot_rmsd_hist(
    _axes[0, 1], prot_rmsd_aligned_arr, "Protein backbone – Kabsch-aligned"
)
plot_rmsd_hist(_axes[1, 0], lig_rmsd_arr, "Ligand heavy-atom – per-atom")
plot_rmsd_hist(
    _axes[1, 1], lig_rmsd_aligned_arr, "Ligand heavy-atom – Kabsch-aligned"
)
_fig.tight_layout()
_fig

```


Evaluated 661 complexes

Protein backbone RMSD – per-atom (Å):

Mean: 1.8810

Median: 1.7793

Std: 0.6184

Protein backbone RMSD – Kabsch-aligned (Å):

Mean: 1.7215

Median: 1.6460

Std: 0.5571

Ligand heavy-atom RMSD – per-atom (Å):

Mean: 3.1646

Median: 2.4634

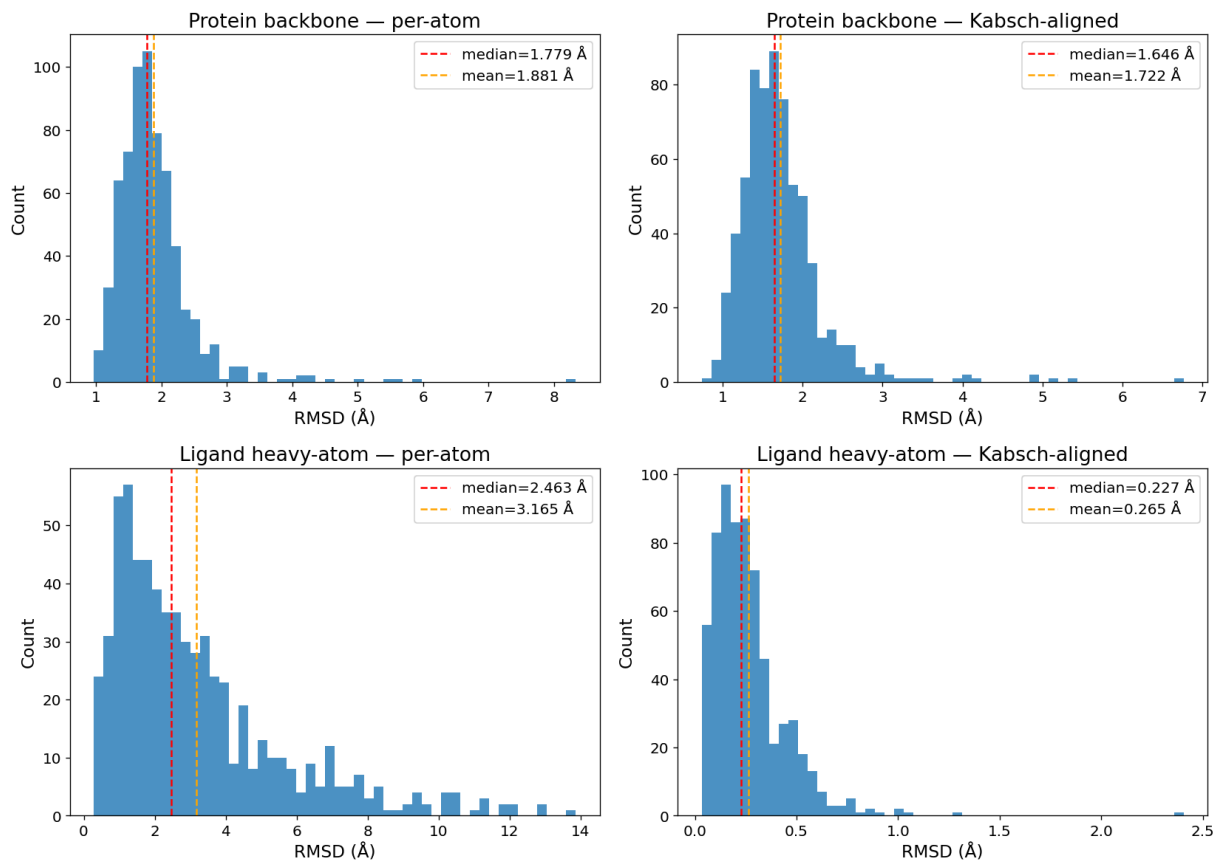
Std: 2.4539

Ligand heavy-atom RMSD – Kabsch-aligned (Å):

Mean: 0.2653

Median: 0.2272

Std: 0.1890



5.6 Complex-aligned RMSD (Å)

タンパク質バックボーン + リガンド重原子を **1つの剛体** として連結し、複合体ごとにもとめて Kabsch アライメントしたあとの RMSD を見る。

- **Whole complex:** アライメント後の全原子 RMSD。複合体ポーズ全体の再現精度。

- **Protein in joint frame:** 同じ joint アライメント後の、タンパク質側だけの残差 RMSD。
- **Ligand in joint frame:** 同じ joint アライメント後の、リガンド側だけの残差 RMSD。タンパク質を基準に固定したときの、リガンドの相対ポーズずれ。

```
In [ ]: def _print_joint_stats(name: str, arr: np.ndarray) -> None:
    print(f"{name}:")
    print(f"  Mean:    {arr.mean():.4f}")
    print(f"  Median:  {np.median(arr):.4f}")
    print(f"  Std:     {arr.std():.4f}")

    _print_joint_stats("Whole-complex RMSD – joint Kabsch (Å)", joint_rmsd_arr)
    print()
    _print_joint_stats(
        "Protein component in joint frame (Å)", prot_in_joint_rmsd_arr
    )
    print()
    _print_joint_stats(
        "Ligand component in joint frame (Å)", lig_in_joint_rmsd_arr
    )

    _fig, _axes = plt.subplots(1, 3, figsize=(18, 5))
    plot_rmsd_hist(_axes[0], joint_rmsd_arr, "Whole complex – joint Kabsch")
    plot_rmsd_hist(
        _axes[1], prot_in_joint_rmsd_arr, "Protein in joint frame"
    )
    plot_rmsd_hist(_axes[2], lig_in_joint_rmsd_arr, "Ligand in joint frame")
    _fig.tight_layout()
    _fig
```

Whole-complex RMSD – joint Kabsch (Å):

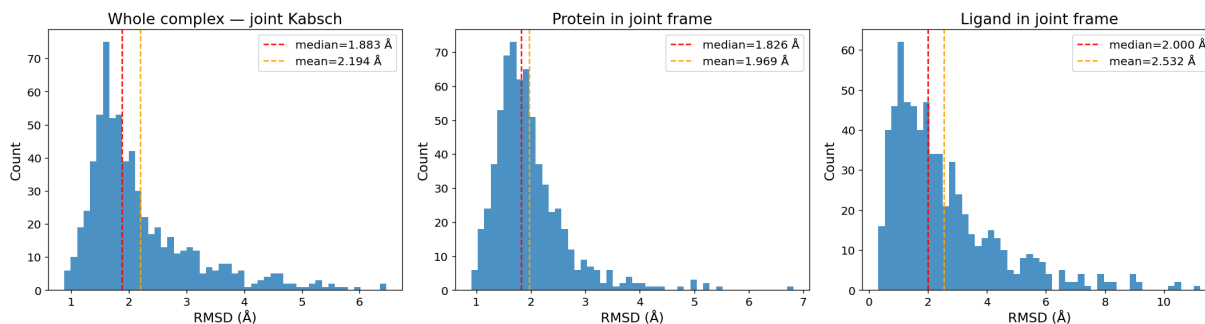
Mean: 2.1944
Median: 1.8828
Std: 0.9474

Protein component in joint frame (Å):

Mean: 1.9687
Median: 1.8264
Std: 0.6806

Ligand component in joint frame (Å):

Mean: 2.5316
Median: 2.0004
Std: 1.8442



6. Latent space visualization (t-SNE)

Codebook ベクトルとエンコーダ出力を t-SNE で2次元に射影し、量子化の質を確認する。

```
In [ ]: from sklearn.manifold import TSNE

def plot_latent_tsne(
    encoder_z: "Tensor",
    codebook: "EMACodebook",
    _codebook_size: int,
    name: str,
    n_samples: int = 5000,
) -> None:
    """t-SNE visualization of encoder outputs and codebook vectors.

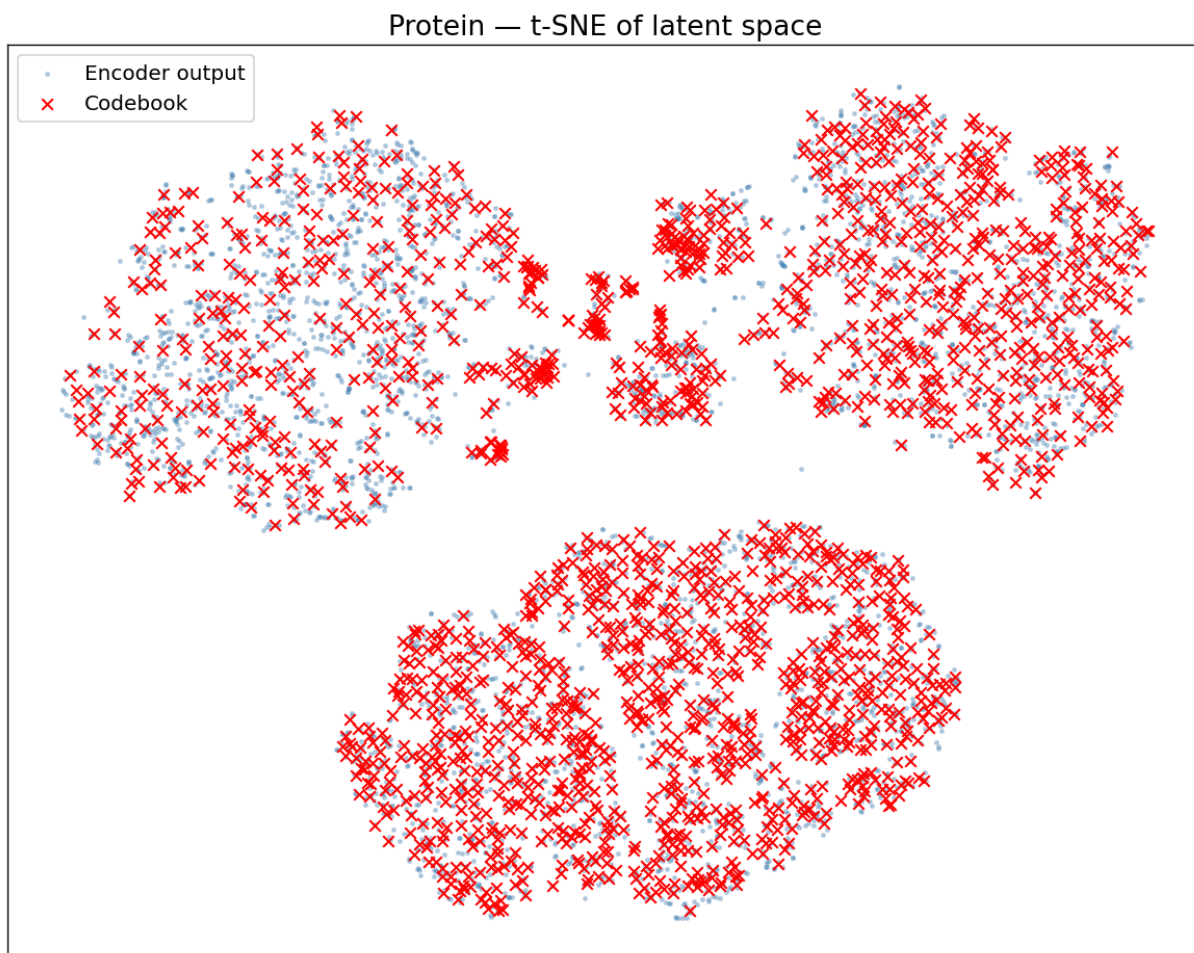
    Vectors are plotted as-is; the codebook does plain L2 nearest-neighbor
    lookup on raw embeddings (encoder z is post-LayerNorm).
    """
    z_arr = encoder_z.cpu().numpy()
    cb_arr = codebook.embedding.cpu().detach().numpy()
    rng = np.random.default_rng(42)
    idx = rng.choice(len(z_arr), min(n_samples, len(z_arr)), replace=False)
    z_sub = z_arr[idx]
    combined = np.vstack([z_sub, cb_arr]) # Subsample encoder outputs
    tsne = TSNE(
        n_components=2, random_state=42, perplexity=min(30, len(combined)) //
    )
    embedded = tsne.fit_transform(combined)
    z_emb = embedded[: len(z_sub)]
    cb_emb = embedded[len(z_sub) :]
    _fig, ax = plt.subplots(figsize=(10, 8)) # Combine for t-SNE
    ax.scatter(
        z_emb[:, 0],
        z_emb[:, 1],
        s=4,
        alpha=0.3,
        c="steelblue",
        label="Encoder output",
    )
    ax.scatter(
        cb_emb[:, 0],
        cb_emb[:, 1],
        s=40,
        c="red",
        marker="x",
        linewidths=1.2,
        label="Codebook",
    )
    ax.set_title(f"{name} - t-SNE of latent space")
    ax.legend()
    ax.set_xticks([])
    ax.set_yticks([])
```

```

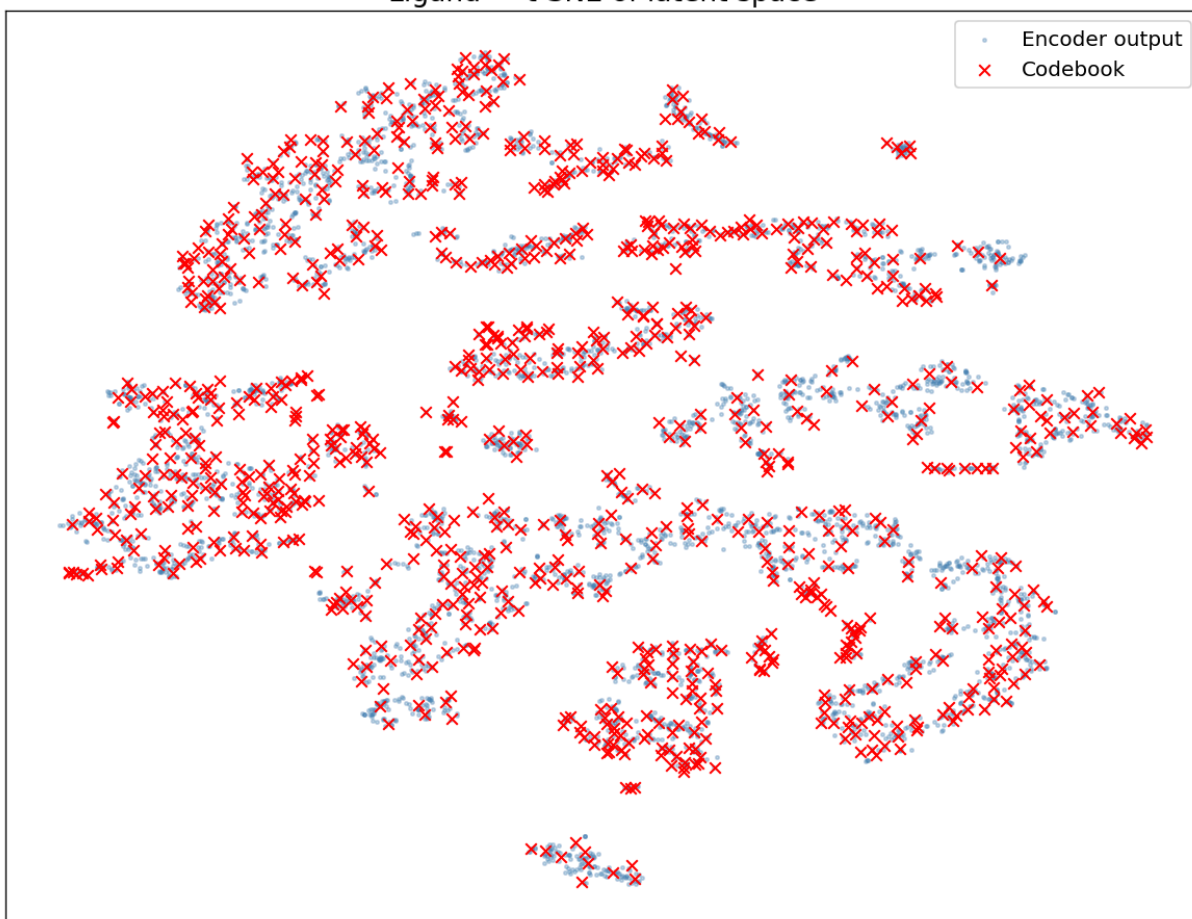
_fig.tight_layout()
plt.show()

plot_latent_tsne(
    prot_z, protein_vqvae.codebook, config.protein.codebook_size, "Protein"
)
plot_latent_tsne(
    lig_z, ligand_vqvae.codebook, config.ligand.codebook_size, "Ligand"
)

```



Ligand — t-SNE of latent space



7. Summary table

```
In [ ]: import pandas as pd

prot_idx_np = prot_indices.cpu().numpy()
lig_idx_np = lig_indices.cpu().numpy()

prot_probs = np.bincount(
    prot_idx_np, minlength=config.protein.codebook_size
).astype(float)
prot_probs = prot_probs / prot_probs.sum()
prot_probs_nz = prot_probs[prot_probs > 0]
prot_ppl = np.exp(-np.sum(prot_probs_nz * np.log(prot_probs_nz)))

lig_probs = np.bincount(lig_idx_np, minlength=config.ligand.codebook_size).a
    float
)
lig_probs = lig_probs / lig_probs.sum()
lig_probs_nz = lig_probs[lig_probs > 0]
lig_ppl = np.exp(-np.sum(lig_probs_nz * np.log(lig_probs_nz)))

summary = pd.DataFrame(
    {
        "Metric": [
            "Codebook size",
```

```

        "Latent dim",
        "Test MSE (normalized)",
        "Test MSE (original scale)",
        "Test RMSE (original scale)",
        "Active codes",
        "Utilization",
        "Perplexity",
        "Perplexity (normalized)",
    ],
    "Protein": [
        config.protein.codebook_size,
        config.protein.latent_dim,
        f"{prot_metrics['mse_norm']:.6f}",
        f"{prot_metrics['mse_orig']:.6f}",
        f"{np.sqrt(prot_metrics['mse_orig']):.6f}",
        len(np.unique(prot_idx_np)),
        f"{len(np.unique(prot_idx_np)) / config.protein.codebook_size:.4f}",
        f"{prot_ppl:.1f}",
        f"{prot_ppl / config.protein.codebook_size:.4f}",
    ],
    "Ligand": [
        config.ligand.codebook_size,
        config.ligand.latent_dim,
        f"{lig_metrics['mse_norm']:.6f}",
        f"{lig_metrics['mse_orig']:.6f}",
        f"{np.sqrt(lig_metrics['mse_orig']):.6f}",
        len(np.unique(lig_idx_np)),
        f"{len(np.unique(lig_idx_np)) / config.ligand.codebook_size:.4f}",
        f"{lig_ppl:.1f}",
        f"{lig_ppl / config.ligand.codebook_size:.4f}",
    ],
}
)

summary

```

	Metric	Protein	Ligand
0	Codebook size	2048	1024
1	Latent dim	16	8
2	Test MSE (normalized)	0.127916	0.017901
3	Test MSE (original scale)	0.086713	0.009797
4	Test RMSE (original scale)	0.294471	0.098979
5	Active codes	2035	1022
6	Utilization	0.9937	0.9980
7	Perplexity	1606.7	759.0
8	Perplexity (normalized)	0.7845	0.7412